POLAR-PIC: A Holistic Framework for Matrixized PIC with Co-Designed Compute, Layout, and Communication

Yizhuo Rao
Sun Yat-sen University
Guangzhou, China
raoyzh6@mail2.sysu.edu.cn

Xingjian Cui*
Sun Yat-sen University
Guangzhou, China
cuixj8@mail2.sysu.edu.cn

Shangzhi Pang
Sun Yat-sen University
Guangzhou, China
pangshzh@mail2.sysu.edu.cn

Jiabin Xie
Sun Yat-sen University
Guangzhou, China
xiejb6@mail2.sysu.edu.cn

Guangnan Feng
Sun Yat-sen University
Guangzhou, China
fenggn7@mail.sysu.edu.cn

Ziyan Zhang
Sun Yat-sen University
Guangzhou, China
zhangzy273@mail2.sysu.edu.cn

Jinhui Wei
Sun Yat-sen University
Guangzhou, China
weijh28@mail2.sysu.edu.cn

Languang Gao
Sun Yat-sen University
Guangzhou, China
gaolg@mail2.sysu.edu.cn

Zhenyu Wang
Institute of Plasma Physics, Chinese Academy of Sciences
Hefei, China
zhenyu.wang@ipp.ac.cn

Zhiguang Chen†
Sun Yat-sen University
Guangzhou, China
zhiguang.chen@nscc-gz.cn

Yutong Lu
Sun Yat-sen University
Guangzhou, China
luyutong@mail.sysu.edu.cn

Abstract

Particle-in-Cell (PIC) simulations are fundamental to plasma physics but often suffer from limited scalability due to particle–grid interaction bottlenecks and particle redistribution costs. Specifically, the particle–grid interaction computations have not taken full advantage of the emerging Matrix Processing Units (MPUs), the particle motion introduces irregular memory accesses, and the bulk-synchronous redistribution further destroys long-term data locality thereby limiting parallel efficiency. To address these inefficiencies, we present **POLAR-PIC**, a co-designed framework for large-scale PIC simulations that (i) reformulates Field Interpolation into an MPU-friendly outer-product form, (ii) maintains a physically ordered particle layout to preserve memory contiguity, and (iii) overlaps particle communication with Deposition to hide redistribution overhead. The evaluation on the pilot system of an Exascale supercomputer demonstrates that **POLAR-PIC** accelerates the entire particle-processing phase by up to 10.9× in uniform plasma and 4.4× in real-world laser-ion acceleration scenarios compared to the native WarpX reference pipeline on LX2. Ablation studies reveal that the speedups achieved by Interpolation and Deposition are 8.0× and 13.2×, respectively, and the asynchronous communication design sustains a 99.1% overlap ratio. In cross-platform comparisons, **POLAR-PIC** achieves 13.2% of theoretical peak efficiency on the CPU-based LS system, while WarpX reaches 9.6% on NVIDIA A800 GPUs. Notably, the scalability evaluation demonstrates that **POLAR-PIC** maintains 67.5% weak scaling efficiency on over 2 million cores under high-migration dynamic workloads, highlighting the importance of holistic co-design for future matrix-centric HPC systems.

CCS Concepts

• **Computer systems organization → Single instruction, multiple data**; • **Computing methodologies → Parallel computing methodologies**.

Keywords

Particle-in-Cell simulation, High-Performance Computing, Scientific Computing, Hardware Acceleration

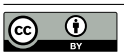
ACM Reference Format:
Yizhuo Rao, Xingjian Cui, Shangzhi Pang, Jiabin Xie, Guangnan Feng, Ziyan Zhang, Jinhui Wei, Languang Gao, Zhenyu Wang, Zhiguang Chen, and Yutong Lu. 2026. POLAR-PIC: A Holistic Framework for Matrixized PIC with Co-Designed Compute, Layout, and Communication. In *The 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '26), July 13–16, 2026, Cleveland, OH, USA.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3806645.3807574

1 Introduction

The Particle-in-Cell (PIC) method is fundamental to kinetic plasma physics and widely used in astrophysical and laboratory plasma research [1, 7, 17, 19]. Large-scale PIC simulations, which track billions of particles while solving Maxwell's equations on Eulerian grids, impose severe pressure on the compute, memory, and interconnect subsystems. On the other hand, as heterogeneous HPC systems evolve toward the Exascale era, efficient hardware utilization has become a critical challenge for both the HPC and computational physics communities.

Although frameworks such as VPIC [7] and WarpX [17, 27] scale well on heterogeneous systems, particle-grid interaction remains

*Contributed equally.
†Corresponding author.



HPDC '26, Cleveland, OH, USA

ACM ISBN 979-8-4007-2640-8/26/07
https://doi.org/10.1145/3806645.3807574

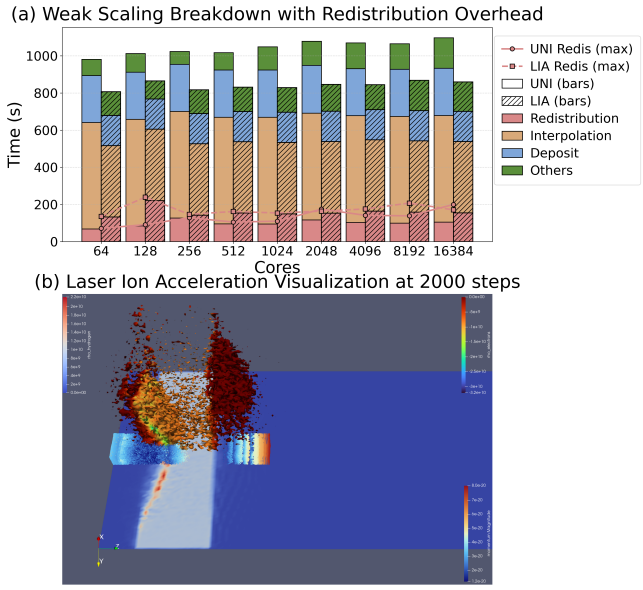


Figure 1: Native WarpX v24.07 Runtime breakdown of Uniform Plasma and Laser-Ion Acceleration simulation on the Tianhe Xingyi platform.

the dominant bottleneck, often accounting for over 80% of runtime [7, 17, 27, 28] (Figure 1). It consists of Field Interpolation, a bandwidth-bound Gather-Stencil kernel, and Charge/Current Deposition, a Scatter-Add kernel limited by write conflicts and atomic overheads [17, 37]. Existing optimizations such as SoA and periodic sorting help [15, 17, 27, 28], but irregular particle motion still prevents sustained bandwidth on VPUs and MPUs.

Specifically, modern processor architectures are rapidly evolving toward matrix-centric compute paradigms. Representative CPUs, incorporating architectural extensions such as Intel AMX [16] and Arm SME [29], now integrate specialized Matrix Processing Units (MPUs). These units execute Matrix Outer-Product Accumulate (MOPA) instructions to compute and accumulate the outer product of two vectors into on-chip tiles within a single cycle, offering a quadratic increase in arithmetic density compared to VPUs. Matrix-PIC [28] capitalizes on this opportunity by reformulating the Deposition kernel into an outer-product form. By combining incremental particle sorting with tile-level buffering to mitigate write conflicts, it achieves significant acceleration.

However, localized optimization of the Deposition phase is insufficient to resolve particle-processing performance bottlenecks in MPU-based PIC simulations, due to three fundamental limitations. First, governed by Amdahl's Law, accelerating Deposition shifts the primary computational bottleneck to the Field Interpolation stage. Mathematically, Field Interpolation constitutes an inner-product reduction (many-to-one), presenting a structural mismatch with the native outer-product mechanism of MPUs (one-to-many). Second, Matrix-PIC[28] maintains only the logical order of particle indices, relying on intermittent global reordering to restore physical memory contiguity. As particle disorder accumulates, the growing reordering cost introduces significant runtime overhead and performance jitter. Third, mainstream frameworks such as WarpX/AMReX [17, 38] typically employ bulk-synchronous parallel (BSP) execution models, treating particle redistribution as a global synchronization point. While these frameworks exhibit robust weak scalability, the BSP pattern constrains single-step performance, particularly in high-dynamic, non-uniform scenarios like Laser-Ion Acceleration (LIA). As shown in Figure 1, the redistribution cost in LIA escalates with increasing parallelism, identifying particle redistribution as an expanding bottleneck at scale.

To address the challenges of operator structural mismatch, fragmented data supply, and blocking synchronization, we propose **POLAR-PIC** (**P**article **O**uter-product, **L**ocality-**A**ware, and **R**MA-overlapped **PIC**). This framework co-designs kernel reformulation, data layout maintenance, and communication scheduling to elevate the optimization scope from isolated kernel acceleration to particle-processing critical path reduction on MPU-based architectures. The primary contributions of this paper are organized as follows:

(1) **We introduce a novel co-design of a matrix outer-product formulation for the PIC Field Interpolation operator.** By transforming the Field Interpolation operator into an MPU-compatible outer-product form via a cell-centric batching strategy, we bridge the structural mismatch between the Interpolation's native inner-product logic and the hardware's outer-product primitives, mitigating the bottleneck shift predicted by Amdahl's Law.
(2) **We propose an inline Sort-on-Write (SoW) mechanism to prevent physical layout degradation arising from the divergence between logical indexing and physical storage.** Rather than performing a periodic global sort, SoW leverages the inherent read-modify-write path and thread-local tile buffering to maintain strict cell-grouped physical contiguity for the majority of particles with negligible amortized $O(1)$ overhead, ensuring a stable, contiguous data supply for matrix-based operations.
(3) **We design a fine-grained communication overlap strategy using pre-packing and notifiable RMA.** To mitigate synchronization stalls caused by particle redistribution, we fuse the packing of migrating particles directly into the Field Interpolation kernel and initiate non-blocking transfers using the high-performance one-sided communication library UNR [18]. This scheduling hides packing overhead, overlaps particle communication with the Deposition kernel, and optimizes the entire particle communication path.

In summary, **POLAR-PIC** unifies outer-product reformulation, SoW-based locality maintenance, and RMA-overlapped communication into a single optimization framework for matrix-centric PIC. Built on WarpX, it accelerates the particle-processing phase by up to 10.9× in uniform plasma and 4.4× in laser-ion acceleration over the native LX2 baseline, while outperforming Matrix-PIC by 4.7× and 3.8×, respectively. For context, POLAR-PIC attains 13.2% of theoretical peak efficiency on the LS pilot system, compared with 9.6% for WarpX on NVIDIA A800.

2 Related Work

2.1 Vectorization and Matrixization in PIC

As processor architectures evolve toward heterogeneous designs, exploiting SIMD techniques is increasingly necessary for maximizing PIC compute throughput. Early architectural optimizations focused on vector processing units (VPUs), leveraging data-layout transformations (AoS-to-SoA) [13], particle binning [13], and portable VPU backends in production codes [7, 17, 27, 38], with adaptive SIMD strategies to accommodate nonuniform particle distributions [15]. Nevertheless, irregular Gather/Scatter behavior remains largely constrained by bandwidth and address-generation overheads, a limitation also observed when mapping sparse/irregular kernels to accelerators [23], while similar refactor-and-vectorize practices have benefited other large-scale scientific codes [20]. The subsequent emergence of matrix-centric hardware has driven a paradigm shift toward mapping structured-grid stencils onto high-density Matrix Outer-Product Accumulate (MOPA) primitives [10, 24, 26, 39], alongside layout-aware redesigns in scientific models [9] and even matrix-centric formulations of compression algorithms [32]. In particle simulations, Matrix-PIC [28] pioneered refactoring the Deposition kernel into an MPU-friendly outer-product formulation on emerging matrix-centric CPUs, opening a CPU-oriented path that is complementary to mature GPU-based PIC pipelines. Rather than targeting a direct replacement of existing GPU solutions, this line of work explores how matrix-oriented computation, locality maintenance, and runtime scheduling can be co-designed within a unified general-purpose execution environment. However, Matrix-PIC leaves Field Interpolation unresolved due to the structural mismatch between Interpolation's inner-product reduction and the native outer-product mechanism of MPUs. Consequently, effectively mapping massive particle Gather to MPUs remains an open gap on next-generation matrix architectures; removing this emerging Amdahl bottleneck is the efficiency gap that this work aims to close.

2.2 Data Layout and Particle Sorting

PIC performance hinges on maintaining spatio-temporal locality against particle diffusion [5, 34], typically necessitating SoA layouts [4, 5, 13] and Space-Filling Curves (SFC) [4]. To counter dynamic locality degradation, production frameworks employ periodic full physical sorting to restore memory compaction [3, 7, 27]. However, this incurs prohibitive $O(N)$ data movement costs that preclude high-frequency execution. Alternatively, lightweight index-based strategies, such as the GPMA in Matrix-PIC [6, 28, 36] or incremental binning [5, 15], reduce movement overheads but may not guarantee physical contiguity, leading to memory fragmentation that starves bandwidth-sensitive MPU pipelines. To resolve this tension between sorting cost and memory contiguity, POLAR-PIC adopts a Controlled Data Duplication mechanism inspired by optimization techniques in sparse matrix and graph processing [2, 23]. This approach preserves the sustained physical contiguity required for matrix acceleration while mitigating the excessive overhead of full global reordering.

2.3 Computation-Communication Overlap

While computation-communication overlap is a canonical strategy for scalability, standard MPI non-blocking primitives often suffer from pseudo-asynchronous behavior due to the lack of independent background progress [12, 14, 22, 25, 40]. To overcome these software stack limitations, Remote Direct Memory Access (RDMA) and One-sided Communication mechanisms have been developed to offload data transfer to hardware, featuring optimizations for intra-node copy engines [11], distributed locks [31], host memory management [33], and event-driven notification as seen in Unified Notifiable RMA (UNR) [18]. However, existing overlap schemes predominantly focus on static grid Halo exchange [21, 30], leaving irregular, data-dependent particle redistribution and its packing/transfer constrained by the end-of-step batching model inherent to bulk-synchronous parallel (BSP) designs. POLAR-PIC fills this gap by embedding particle packing and transmission into the computational data path, thereby avoiding explicit end-of-step synchronization on the critical path and reducing tail effects caused by migration-related overheads.

3 Preliminaries

This section formalizes the mathematical model of Field Interpolation and analyzes its architectural bottlenecks to motivate the proposed MPU-based optimization and SoW design.

3.1 Field Interpolation and Particle Update

Field Interpolation maps electromagnetic fields from discrete grid points to continuous particle positions. Mathematically, this operation represents a high-dimensional tensor contraction process, where the weight tensor derived from shape functions is contracted with the grid field tensor along the stencil dimension to resolve the field value at particle coordinates.

For a particle located at $\mathbf{x}_p = (x, y, z)$, the interpolated physical quantity F_p (such as E_p or B_p) is derived by weighting grid node values within the local support stencil. Assuming a shape function of order S, we have:

$$F(\mathbf{x}_p) = \sum_{k=0}^{S} \sum_{j=0}^{S} \sum_{i=0}^{S} \left(S_x(i)\, S_y(j)\, S_z(k) \right) \cdot F_{\mathbf{grid}}(i_0 + i,\ j_0 + j,\ k_0 + k). \tag{1}$$

where S_x, S_y, S_z denote the normalized shape factors along the axes, and (i_0, j_0, k_0) represents the base index of the grid cell containing the particle. The order S dictates the stencil width, necessitating that the summation indices i, j, k traverse all $S+1$ spatially coupled grid nodes starting from the base anchor (i_0, j_0, k_0).

While Field Interpolation constitutes a memory-bound gather-stencil bottleneck on next-generation architectures due to its structural mismatch with MPU primitives, the subsequent **Particle Push** phase exposes an unavoidable write-back path that we leverage for layout maintenance. Using the interpolated fields $\mathbf{E}_p$ and $\mathbf{B}_p$, this kernel updates particle momentum and position via the Lorentz force equation [8, 35]. Crucially, the push operation follows a deterministic *read-modify-write* dataflow: it streams current attributes to compute the next state and commits the results to memory. This mandatory write-back path serves as the pivotal leverage point for

the layout optimizations proposed in this work, enabling dynamic reordering without additional memory passes.

3.2 Matrix Outer-Product Calculation Paradigm

To overcome the scaling limitations of traditional vector architectures, recent CPUs increasingly integrate Matrix Processing Units (MPUs). Distinct from Vector Processing Units (VPUs), MPUs natively support the Matrix Outer-Product Accumulate (MOPA) operation. Given $\mathbf{a} \in \mathbb{R}^m$ and $\mathbf{b} \in \mathbb{R}^n$, MOPA computes their outer product and accumulates it into an on-chip 2D tile register $\mathbf{C} \in \mathbb{R}^{m \times n}$:

$$\mathbf{C} \leftarrow \mathbf{C} + \mathbf{a} \otimes \mathbf{b}. \tag{2}$$

Representative implementations like Arm SME deliver a near-quadratic increase in compute density by executing $m \times n$ fused multiply-add updates operations per instruction. However, the irregular access patterns inherent to PIC simulations impede pipeline saturation, necessitating paradigm migration and locality optimizations to regularize data supply for effective MPU utilization.

3.3 PIC Workflow and Redistribution Overheads

As illustrated in the central loop of Figure 2, a standard electromagnetic PIC timestep cycles through four distinct phases: *Interpolation & Push*, *Deposition*, *Field Solve*, and *Particle Redistribute*.

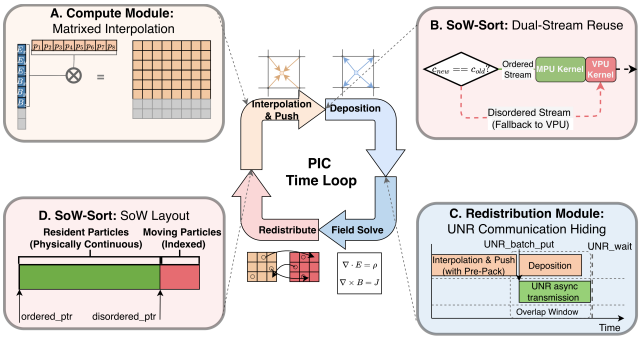


Figure 2: POLAR-PIC integrates the MPU-based Interpolation, the Dual-Stream Reuse, the UNR-based asynchronous communication pipeline, and SoW Layout.

Conventionally (e.g., in WarpX [17]), the Particle Redistribute phase is positioned at the end of the timestep as a bulk-synchronous parallel (BSP) barrier. This process sequences three steps: scanning and packing boundary-crossing particles, blocking communication, and unpacking. In high-dynamic scenarios, the substantial overhead of particle packing combined with this blocking communication pattern becomes a dominant performance bottleneck.

3.4 UNR Asynchronous Communication Library

The Unified Notifiable RMA (UNR) library [18] provides a lightweight asynchronous interface for HPC, offering one-sided RMA built on low-level communication primitives and enhanced completion notification: `UNR_Put/Get` offload transfers to the NIC via RDMA with minimal CPU involvement, while notifiable completion uses native event counters to signal data arrival without explicit handshakes or matching, jointly enabling high-performance asynchronous communication suitable for tight compute pipelines in PIC simulations.

4 Methodology

This section details the system design of POLAR-PIC. Guided by the characteristics of next-generation architectures, we propose a co-optimization scheme that integrates an outer-product Field Interpolation operator, a Sort-on-Write (SoW) layout maintenance strategy, and a computation-communication overlap scheduler to collectively address the challenges of compute throughput, memory locality, and communication overheads in PIC simulations.

4.1 Overall Framework

POLAR-PIC adopts an *algorithm-memory-communication co-design* philosophy that unifies coupled computation, data organization, and particle migration into a dataflow pipeline. As illustrated in Figure 2, the framework comprises three interdependent modules.

First, the compute module reformulates the Field Interpolation for MPUs. This module processes particles in cell-based batches, transforming the original per-particle inner-product reduction into a 2D outer-product accumulation form. This module relies on an ordered particle layout to prevent irregular gather-load patterns from diminishing MPU throughput.

Second, SoW maintains the ordered layout and prepares migrating particles for communication. It leverages the particle write-back path to keep resident particles contiguous in memory, while identifying and packing cross-domain particles on the fly, effectively producing the outgoing data needed by redistribution.

Third, the redesigned redistribution module overlaps particle communication with Deposition. After the Field Interpolation phase, it issues non-blocking `UNR_Put` operations to asynchronously transfer the migrant particles packed by SoW. By invoking `UNR_Wait` after the Deposition kernel completes, the primary communication latency is effectively hidden within the computation pipeline, leaving only a minimal merge step at the end of the timestep.

In summary, high computational throughput hinges on the ordered in-memory layout maintained by SoW, while SoW also packs migrating particles for the redistribution module to overlap transmission and collection. This co-design enables POLAR-PIC to achieve particle-processing phase performance improvements beyond isolated kernel optimizations on next-generation HPC platforms.

4.2 Matrixized Field Interpolation

Field Interpolation projects electromagnetic fields from the Eulerian grid onto Lagrangian particle positions and constitutes a typical *gather-stencil* kernel. As illustrated in Figure 3, each particle performs a weighted accumulation over K stencil nodes, yielding low operational intensity and an inherent many-to-one inner-product reduction between a weight vector and a field vector. To bridge this structural mismatch with MPU-style outer-product execution, POLAR-PIC introduces an outer-product reformulation via **Batching through Tensor Stacking**. By logically stacking N particles

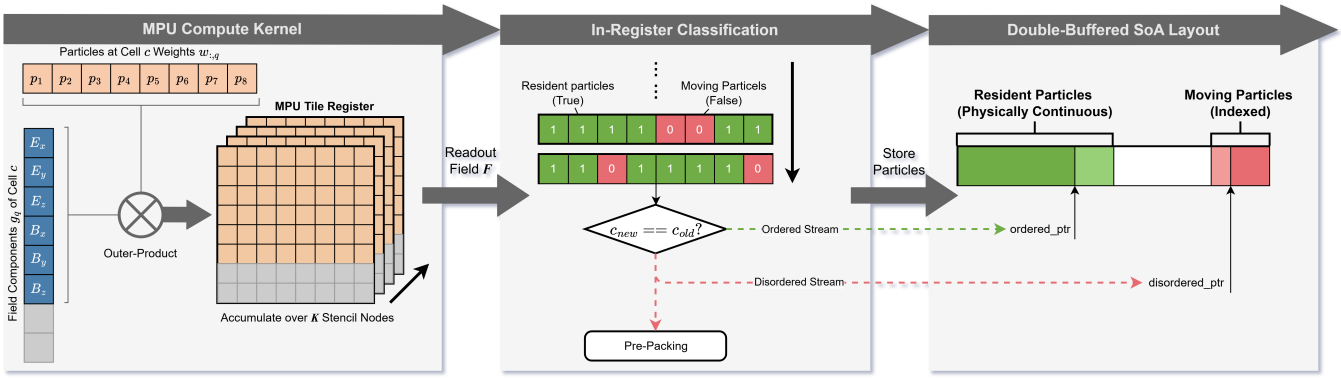


Figure 3: Matrixized Field Interpolation and Sort-on-Write dataflow. Left: Matrix outer-product Interpolation via tensor stacking; Middle: in-register classification and pre-packing; Right: double-buffered SoA layout that maintains a physically contiguous layout.

within the same cell (e.g., $N = 8$ in FP64), it casts N independent $1 \times K$ linear contractions into a single $N \times K$ matrix form, thereby shifting the kernel from a 1-D vector pipeline to an MPU-friendly 2-D batched dataflow that exposes outer-product accumulation. Since typical particle densities far exceed the batch size, full batches are efficiently formed from contiguous SoA segments. Residual particles fewer than N are handled via zero-padding, ensuring a unified matrix execution path.

Let K denote the stencil size for 3-D Interpolation (e.g., $K = 4^3 = 64$ for 3rd-order Interpolation) and D denote the dimension of field components (typically $\{E_x, E_y, E_z, B_x, B_y, B_z\}$). For a particle p, the interpolated field $\mathbf{f}_p \in \mathbb{R}^D$ is accumulated from K stencil nodes:

$$\mathbf{f}_p = \sum_{q=1}^{K} w_{p,q} \cdot \mathbf{g}_q, \tag{3}$$

where $\mathbf{g}_q \in \mathbb{R}^D$ is the field vector at grid node q, and $w_{p,q}$ is the geometric weight scalar for particle p at that node (i.e., the product of shape functions $S_x S_y S_z$).

By stacking N particles from the same cell, we construct a weight matrix $\mathbf{W} \in \mathbb{R}^{N \times K}$ and a grid-field matrix $\mathbf{G} \in \mathbb{R}^{K \times D}$:

$$\mathbf{W} = \begin{bmatrix} \mathbf{w}_{p_1}^{\mathsf{T}} \\ \vdots \\ \mathbf{w}_{p_N}^{\mathsf{T}} \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} \mathbf{g}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{g}_K^{\mathsf{T}} \end{bmatrix}, \tag{4}$$

where each $\mathbf{w}_{p_i} \in \mathbb{R}^K$ stacks the stencil weights of particle p_i. The Interpolation result for the entire batch, $\mathbf{F} \in \mathbb{R}^{N \times D}$, can be expressed as the matrix multiplication $\mathbf{F} = \mathbf{WG}$. Expanding along the stencil dimension yields a sum of *rank-1 updates*:

$$\mathbf{F} = \sum_{q=1}^{K} \left(\mathbf{w}_{:,q} \otimes \mathbf{g}_q \right), \tag{5}$$

where $\mathbf{w}_{:,q} \in \mathbb{R}^N$ is the stacked weight vector of N particles for stencil node q, and $\mathbf{g}_q \in \mathbb{R}^D$ represents the corresponding field-component vector. This form maps directly to the MPU dataflow. The loop iterates over stencil nodes, loads one $\mathbf{g}_q$, and accumulates $\mathbf{w}_{:,q} \otimes \mathbf{g}_q$ into a tile register via MOPA. For a field vector with $D = 6$, $\mathbf{g}_q$ can be zero-padded to the tile width, leading to the tile update:

$$\mathbf{F}_{\text{tile}} \leftarrow \mathbf{F}_{\text{tile}} + \mathbf{w}_{:,q} \otimes \mathbf{g}_q. \tag{6}$$

4.3 Sort-on-Write Algorithm

The performance of MPU-based Interpolation hinges on contiguous data supply, which traditional index-based sorting [28] fails to guarantee. To resolve the memory fragmentation bottleneck, we propose the **Sort-on-Write (SoW)** mechanism. As shown in Figure 3 and Algorithm 1, SoW exploits the *read-modify-write* dataflow of particle push to transform explicit reordering into an **inlined streaming split**. This design dynamically maintains a physically contiguous layout without a periodic stop-the-world global sort.

4.3.1 Dual-Region Memory Management. Leveraging the CFL condition that most particles remain local within a time step, we partition each tile's memory into a dominant **Ordered Region** (physically contiguous) and a small **Disordered Region** (append-only tail). This dual-region layout is maintained using two pre-allocated buffers per rank-local tile, sized by a runtime upper-bound heuristic to avoid reallocation during timesteps while keeping the extra space cost local to each tile. Runtime pointer swapping reuses these buffers across timesteps, thereby eliminating frequent dynamic allocation overheads without resorting to full-volume global double-buffering. Within this layout, the Ordered Region supports direct MPU vector loading, whereas the Disordered Region relies on a lightweight auxiliary index (`bin_to_ip`) to recover logical traversal order. This hybrid organization keeps the vast majority of memory accesses contiguous while confining fragmentation to a small tail. During the SoW process, those disordered particles are progressively absorbed back into the Ordered Region of the next buffer, naturally restoring physical locality over time.

4.3.2 Execution Flow and Layout Reuse. The SoW execution follows a fused pipeline (Algorithm 1):

- **Tail Sorting:** At the beginning of each time step, we perform a low-cost $O(N_{tail})$ binning pass on the Disordered Region to enable logical-order traversal.

Algorithm 1 SoW-Enabled Time-Step: Stream-Split, Comm-Fusion, and Layout Reuse

Require: Particles $\mathcal{P}_{cur}$ (Ordered + Indexed), Fields **E**, **B**, Comm Buffers $\mathcal{B}_{local}$, $\mathcal{B}_{remote}$, Pre-defined Masks $\mathbf{m}_{stay}$, $\mathbf{m}_{move}$, $\mathbf{m}_{rmt}$
Ensure: Updated $\mathcal{P}_{next}$ partitioned for reuse; Comm initiated; Current/Charge **J** deposited

1: *// Disordered particles in $\mathcal{P}_{cur}$ are pre-binned for logical traversal*
2: Initialize cursors: $ptr_{ord} \leftarrow 0$, $ptr_{dis} \leftarrow$ Capacity($\mathcal{P}_{next}$);
// Phase 1: Interpolation & Push with SoW
3: **for** each cell $c \in$ Tile T **do**
4: Meta[c].start $\leftarrow ptr_{ord}$; ▷ start offset for next frame
5: $\mathcal{S}_{ord} \leftarrow$ **VPU_Load**($\mathcal{P}_{cur}$.Ordered[c]);
6: $\mathcal{S}_{idx} \leftarrow$ **VPU_GatherLoad**($\mathcal{P}_{cur}$.Disordered, bin_to_ip[c]);
7: **for** batch $\mathbf{P}_{in} \in$ ($\mathcal{S}_{ord}, \mathcal{S}_{idx}$) **do**
8: $\mathbf{P}_{new} \leftarrow$ **MPU_InterpolationKernel**($\mathbf{P}_{in}$, **E**, **B**) ;
// In-register Classification & Stream-Split
9: $\mathbf{c}_{new} \leftarrow$ **GetCellID**($\mathbf{P}_{new}$);
10: $\mathbf{m}_{stay} \leftarrow (\mathbf{c}_{new} == c)$;
11: $\mathbf{m}_{move} \leftarrow (\neg \mathbf{m}_{stay})$;
// Keep Resident physically contiguous
12: **CompactStore**($\mathcal{P}_{next}[ptr_{ord}]$, $\mathbf{P}_{new}$, $\mathbf{m}_{stay}$) ;
13: $ptr_{ord} \leftarrow ptr_{ord} +$ **PopCount**($\mathbf{m}_{stay}$);
14: **if** Any($\mathbf{m}_{move}$) **then**
15: $N_{move} \leftarrow$ **PopCount**($\mathbf{m}_{move}$);
// Grow Disordered Region from Tail
16: $ptr_{dis} \leftarrow ptr_{dis} - N_{move}$;
17: **CompactStore**($\mathcal{P}_{next}[ptr_{dis}]$, $\mathbf{P}_{new}$, $\mathbf{m}_{move}$);
// Fusion: Pre-pack based on Destination
18: $\mathbf{m}_{rmt} \leftarrow$ **IsRemoteRank**($\mathbf{P}_{new}, \mathbf{m}_{move}$);
19: **CopyToBuffer**($\mathcal{B}_{local}$, $\mathbf{P}_{new}$, $\mathbf{m}_{move} \wedge \neg \mathbf{m}_{rmt}$);
20: **CopyToBuffer**($\mathcal{B}_{remote}$, $\mathbf{P}_{new}$, $\mathbf{m}_{rmt}$);
21: **end if**
22: **end for**
23: Meta[c].length $\leftarrow ptr_{ord}$ − Meta[c].start
24: **end for**
25: **TriggerUNR_BatchPut**($\mathcal{B}_{remote}$, PopCount($\mathbf{m}_{rmt}$));
// Phase 2: Deposition with Layout Reuse
26: **for** each cell $c \in$ Tile T **do** ▷ Reuse ordered layout
27: $\mathbf{P}_{ord} \leftarrow$ VPU_Load($\mathcal{P}_{next}$, Meta[c].start, Meta[c].length)
28: **MPU_DepositKernel**($\mathbf{P}_{ord}$, **J**)
29: **end for**
30: **for** particle $p \in$ Range(ptr_{dis}, Capacity) **do** ▷ Fallback to VPU
31: **VPU_DepositKernel**(p, **J**)
32: **end for**

- **Stream-Split Write-back (Lines 9-22):** During the update phase, SIMD instructions classify particles as *resident* or *migrating*. A dual-pointer streaming store then separates them: resident particles are compacted into the Ordered Region of the next buffer, while migrating particles are appended to the Disordered tail.

This mechanism acts as a self-healing process, continuously absorbing disordered particles back into the ordered layout. Furthermore, since position updates preserve cell indices for the duration of the step, the **Deposition** kernel (Lines 26-30) directly reuses this contiguous layout for high-throughput MOPA execution, falling back to VPU atomics only for the sparse tail.

4.4 Overlapping Particle Communication with Deposition

From a dataflow perspective, the updated particle position $\mathbf{x}^{n+1}$ is available at the end of *Particle Push*, and subsequent Deposition and field update do not change the particle's spatial mapping to grid indices. Accordingly, redistribution can be partially initiated early by (i) determining the destination tile/rank from $\mathbf{x}^{n+1}$ and (ii) pre-packing a copy of migrating particle attributes into pre-allocated remote send buffers or local tiles. As illustrated in Figure 4, this design transforms the traditional blocking communication phase into an asynchronous pipeline, effectively masking transmission latency while avoiding interference with field solvers.

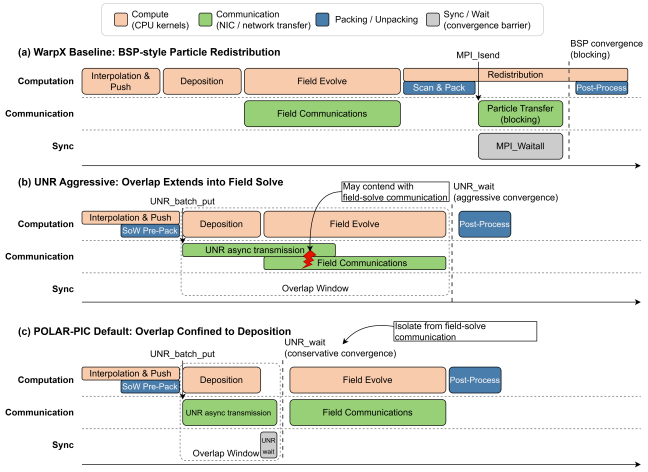


Figure 4: Comparison of Particle Redistribution Strategies.

4.4.1 Re-packing via Kernel Fusion. In native WarpX redistribution, migrant particles are identified and packed through a separate end-of-step scan, which introduces redundant traversal and additional memory traffic. POLAR-PIC removes this standalone scan-and-pack stage by fusing pre-packing into the write-back path, where SoW already exposes the destination information of migrated particles. The runtime precomputes neighbor connectivity under the static domain decomposition and registers communication buffers during initialization; thread-private send/recv buffers are used to reduce synchronization and avoid consolidating per-thread outputs.

During write-back, migrants are dispatched directly to their destinations. Intra-rank migrants are written from registers into the target tile's receive buffer, while inter-rank migrants are packed into the thread-private UNR send buffer for the corresponding neighbor. Meanwhile, particles leaving the current tile are swapped into the tail disordered region so they can be removed efficiently at the end of the step. Buffers follow a linear `[Header | Payload]` layout with lightweight per-thread offsets, and UNR batch transmission avoids extra process-level `memcpy` and cache pollution. Importantly, this design does not imply that all packing work disappears; rather,

the remaining packing work is absorbed into the write-back path instead of appearing as a separate end-of-step stage.

4.4.2 Asynchronous Transmission with UNR. After pre-packing completes, the main thread invokes UNR `batch_put` outside the OpenMP region to submit all RDMA writes across threads and neighbor directions in one batch. The NIC then transfers data from thread-local send buffers into remote receive buffers, while the CPU proceeds immediately to the Deposition kernel, thereby overlapping communication with Deposition. A receiver process only waits for completion signals via `UNR_Wait` to detect data arrival, rather than posting receive-side synchronization operations such as `MPI_Recv`, `MPI_Irecv/Wait`, or `MPI_Win_Fence`.

4.4.3 Convergence and Particle Finalization. POLAR-PIC confines overlap to the Deposition window and enforces convergence with `UNR_Wait` immediately after Deposition to avoid interference with the latency-sensitive field-solve communication. This separation reduces contention in NIC queues and network injection bandwidth and improves timestep stability; in practice, most particle transfers complete before Deposition ends, making the post-Deposition wait negligible.

After convergence, received particles are unpacked and finalized. Since SoW clusters invalid entries at the tail disordered region, deletion reduces to tail truncation by updating the SoA size pointer. Incoming `[Header | Payload]` segments are then unpacked and appended to the target tile's SoA tail using parallel `memcpy`. Overall, POLAR-PIC refactors redistribution from an end-of-step blocking BSP phase into a pipelined overlap scheme that achieves effective masking while isolating bandwidth interference.

5 Experimental Setup

5.1 Experimental Platform

Our primary experiments were conducted on the LS pilot system, a next-generation HPC cluster. Each compute node incorporates two LX2 high-performance CPUs. As illustrated in the architectural diagram in Figure 5, each processor package contains over 256 cores distributed across two compute dies. These cores support both vector (VPU) and matrix (MPU) computation engines. The VPUs execute double-precision (FP64) SIMD instructions, while the MPUs are designed for 8×8 matrix operations. Critically, the MPU's MOPA instruction offers approximately 4× the theoretical FP64 performance of the VPU's Multiply-Accumulate (MLA) instruction, presenting a significant opportunity for acceleration. Both compute units operate at frequencies at or above 1.3 GHz.

The LX2 CPU implements a system-on-chip design where each Die features 128GB of off-die DDR memory organized across 4 NUMA domains. The Dies are interconnected through an LXLink network, which provides up to 48 GB/s bidirectional bandwidth per NIC and supports RDMA operations. The LS system employs a customized Linux-based OS with LLVM-based Clang/Flang compilers. All binaries are compiled with `-O3` and `-flto` and linked against an optimized OpenMPI library and architecture-tuned math kernels. For one-sided transfers, we utilize the latest UNR library tuned for LS RDMA verbs. For comparison, the MPI-based redistribution is implemented using non-blocking `MPI_Isend/MPI_Irecv` primitives to provide asynchronous progress.

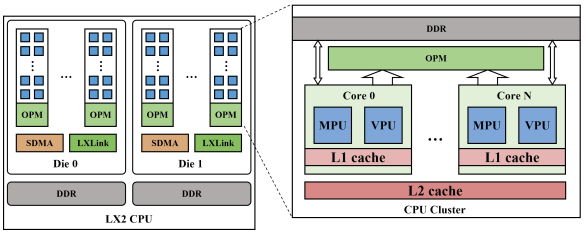


Figure 5: Architectural diagram of the LX2 processor, illustrating the dual-Die design, core distribution, and key components like the VPU, MPU, and memory interfaces.

For cross-platform comparison, we conducted experiments on a platform equipped with two 28-core CPUs and 8 NVIDIA A800 GPUs, which features 80 GB of HBM2e memory. The software environment used for these tests included NVIDIA driver version 535.104.12.

5.2 Workloads and Configuration

We implement the **POLAR-PIC** framework atop the widely adopted `WarpX v24.07` [17]. While our current implementation targets WarpX, the proposed architectural optimizations are generic and portable to other standard PIC workflows. Performance comparisons and ablation studies are conducted on 2 compute nodes using a hybrid MPI+OpenMP parallelization strategy. We configure one MPI rank per NUMA domain (aggregating 16 ranks per node), spawning 32 OpenMP threads per rank with strict thread affinity enforced to minimize context switching overhead. For large-scale weak scalability experiments, we utilize the Uniform Plasma benchmark on LS pilot system, scaling up to 4,096 nodes with the same per-node configuration.

We disable I/O to isolate computational performance. All simulations employ a 3D Cartesian grid with 3rd-order B-spline shape factors, the Yee field solver, direct current Deposition, and the Boris particle pusher. We execute 100 timesteps for measurement after an initial warm-up phase. We use two workloads: (i) a **Uniform Plasma** microbenchmark with a grid of $256 \times 128 \times 128$, where we sweep the particle density, i.e., particles per cell (PPC), to vary computational intensity and memory pressure, and adjust the thermal velocity u_{th} to induce different levels of particle migration; and (ii) a **Laser-Ion Acceleration with a Planar Target** production case with a grid of $192 \times 192 \times 256$ over an approximately $7.5\,\mu m \times 7.5\,\mu m \times 15\,\mu m$ domain to validate particle-phase performance under strongly non-uniform particle distributions and significant migration. All experiments are repeated three times excluding warm-up overhead; we report the average execution time across these trials to filter out transient system jitter. Detailed physical and numerical parameters are listed in Appendix B.

5.3 Evaluation Metrics

To isolate the architectural impact on particle-grid interactions, we define overall **Particle-Phase Time** as $T_{particle} = T_{Interpolation} + T_{deposit} + T_{redistribute}$, where $T_{Interpolation}$ includes Field Interpolation, Push, SoW, and fused re-packing overheads; $T_{deposit}$ includes current Deposition overhead; and $T_{redistribute}$ captures particle

communication issuing, explicit waiting, and post-processing. Furthermore, $T_{steps} = T_{particle}/N_{steps}$ denotes the **average particle-phase time per step**. Unless otherwise specified, all derivative metrics in this work (including speedups, throughput, and efficiency) are normalized against $T_{particle}$ or T_{steps} to exclude the Eulerian field solver and global halo exchanges, which remain identical to the native WarpX pipeline on LX2. Our primary optimization target is therefore the particle-processing critical path, while end-to-end full-timestep behavior is reported separately in the weak-scaling study.

For ablations, we further decompose computation into T_{prep} (index / shape-factor preparation), T_{sort} (IncrSort or SoW overhead), T_{kernel} (pure math execution), and T_{reduce} (write-back / reduction); and we **decompose communication** into T_{pack} (serialization and packing), T_{issue} (issuing MPI / UNR calls), T_{wait} (explicit synchronization), and $T_{post\_process}$ (unpacking, appending, and invalid-particle removal).

To evaluate particle processing throughput and computational density, we report **Particles per Second** (PPS, N_{total}/T_{steps}) and **Cycles per Particle** (CPP), the latter being normalized to a 1.3 GHz reference frequency. The **Effective Transport Overlap Ratio** η_{overlap} is defined as:

$$\eta_{\text{overlap}} = 1 - \frac{T_{issue}^{\text{Overlap}} + T_{wait}^{\text{Overlap}}}{T_{issue}^{\text{baseline}} + T_{wait}^{\text{baseline}}} \tag{7}$$

where the numerator represents the explicit communication time under overlap, and the denominator is the exposed particle-communication time of the BSP-style native `WarpX` redistribution path. Since the BSP approach prohibits overlap, η_{overlap} quantifies the fraction of communication latency removed from the critical path relative to synchronous execution. In the results, we discuss residual synchronization through the communication-critical path and report a representative maximum per-rank wait time

Finally, we evaluate hardware utilization and particle-phase quality. **Peak efficiency** is defined as:

$$\eta_{\text{peak}} = \max\left(\frac{\sum \text{Particle FLOPs}}{T_{steps} \cdot P_{\text{theoretical}}}\right) \times 100\%, \tag{8}$$

where $P_{\text{theoretical}}$ is the theoretical FP64 peak of the platform. Particle FLOPs are standardized using the native WarpX implementation, counting 1,636 and 419 FLOPs per particle for the Interpolation and Deposition kernels, respectively.

Following standard PIC community practice [17], we also report the **per-node Figure of Merit** (FOM_{node}):

$$\text{FOM}_{\text{node}} = \max\left(\frac{\alpha N_c + \beta N_p}{T_{steps} \cdot N_{nodes}}\right), \tag{9}$$

where N_c and N_p are the total number of grid cells and macroparticles, respectively. The weights $\alpha = 0.1$ and $\beta = 0.9$ are adopted to ensure consistency with WarpX benchmarks [17].

5.4 Experimental Design

Table 1 summarizes the ablation variants and integrated system configurations. Here C0 corresponds to the BSP-style WarpX particle-redistribution path without communication-computation overlap. For sorting-based comparisons, Matrix-PIC serves as the state-of-the-art reference, representing the logical-order optimization path beyond the native WarpX baseline. We bold the default POLAR-PIC setup, and provide implementation details in Appendix A.

Table 1: Summary of experimental configurations and ablation variants.

Variant	Key Configuration Characteristics
Exp 1: Interpolation (Gather & Push) Variants	
G0 (Baseline)	WarpX native, Compiler auto-vectorization (VPU), Unsorted
G1	Hand-tuned VPU intrinsics, Unsorted
G2	VPU, Logical Index-Sort (Reproduces Matrix-PIC [28])
G3	VPU, Physical Reordering (based on G2 indices)
G4	VPU, Sort-on-Write (SoW) Physical Reordering
G5	MPU, Logical Index-Sort (MPU version of G2)
G6	MPU, Physical Reordering (MPU version of G3)
G7 (POLAR-PIC)	**MPU, SoW Physical Reordering**
Exp 2: Deposition (Scatter) Variants	
D0 (Baseline)	WarpX native, Compiler auto-vectorization (VPU), Unsorted
D1	MPU, Reuse G2's Logical Index (Reproduces Matrix-PIC [28])
D2	MPU, Reuse G7's Physical Layout + Tail Binning
D3 (POLAR-PIC)	**MPU, Reuse G7's Physical Layout + VPU Tail**
Exp 3: Overlap Strategy Variants	
C0 (Baseline)	No Overlap (BSP-style blocking redistribution)
C1	MPI-based, Conservative sync (Post-Deposit)
C2 (POLAR-PIC)	**UNR-based, Conservative sync (Post-Deposit)**
C3	MPI-based, Aggressive sync (Post-FieldSolve)
C4	UNR-based, Aggressive sync (Post-FieldSolve)
Integrated Systems	
WarpX-Native (Baseline)	Native `WarpX v24.07` reference pipeline on LX2 (G0 + D0 + C0)
Matrix-PIC	State-of-the-art logical-order pipeline (G2 + D1 + C0)
POLAR-PIC	**Full Outer-Product Pipeline (G7 + D3 + C2)**

6 Experimental Results

6.1 Overall Performance

Unless otherwise stated, the results in this subsection focus on particle-phase performance, which is the primary optimization target of POLAR-PIC. End-to-end full-timestep behavior is reported separately in the weak-scaling study in Section6.6.

6.1.1 Uniform Plasma Microbenchmarks. We first evaluate peak particle throughput and dynamic robustness by sweeping particle density (PPC) and thermal velocity (u_{th}). Relative to the native WarpX reference pipeline on LX2, **POLAR-PIC** achieves up to 10.9× particle-phase speedup in high-density regimes and still delivers a 7.3× advantage under high migration intensity; it also outperforms Matrix-PIC by up to 4.7×.

As illustrated in Figure 6, the WarpX-Native reference is bounded by the throughput ceiling of the VPU path. While Matrix-PIC delivers significant speedups at moderate densities, its performance degrades at high PPC (> 256) due to escalating index-maintenance overheads. **POLAR-PIC** avoids this scalability pitfall by relying on the SoW mechanism, preserves physical contiguity and yields a near-monotonic throughput trend as PPC increases. At very low densities ($PPC = 1$), speedups naturally diminish for matrix-based variants because padding and metadata costs are hard to amortize when batches are underfilled, consistent with prior observations on low-occupancy matrix kernels [28].

When sweeping thermal velocity (u_{th}) to emulate dynamic disorder, the advantages of our physical layout management become even more pronounced. While the Baseline shows limited sensitivity

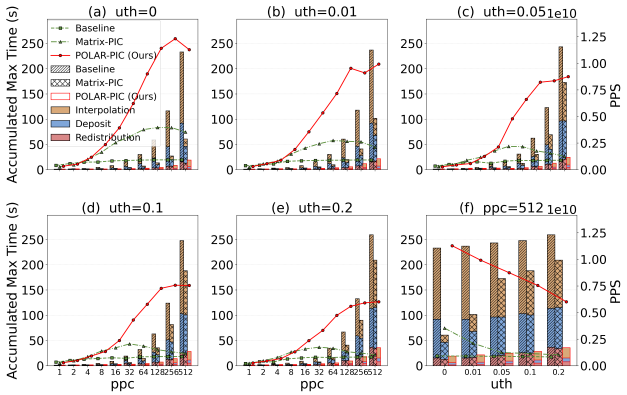


Figure 6: Uniform-plasma particle-phase performance under varying particle density (PPC) and thermal velocity u_{th}.

to disorder because it does not exploit locality, sorting-based methods face challenges as disorder increases. Specifically, at $u_{th} = 0.2$, Matrix-PIC's speedup collapses to just 1.2× due to severe data supply stalls. Conversely, POLAR-PIC demonstrates superior resilience via the SoW mechanism. Even under this worst-case turbulence, it sustains a 7.3× speedup over the Baseline, maintaining contiguous SoA segments for the MPU where logical ordering fails.

6.1.2 Laser-Ion Acceleration Benchmark. In the highly non-uniform LIA benchmark, **POLAR-PIC** sustains a robust 4.4× particle processing speedup over the Baseline and 3.8× over Matrix-PIC, significantly outperforming logical-ordering alternatives in migration-heavy scenarios.

As shown in Figure 7, the performance advantage stems primarily from the optimized handling of particle redistribution, which emerges as the dominant cost in this regime. POLAR-PIC addresses this bottleneck through its asynchronous pipeline, accelerating the redistribution phase by 3.0× relative to the Baseline. Crucially, the analysis of long-tail latency highlights a major stability gap. Matrix-PIC suffers from severe performance jitter due to frequent index rebuilding, with the (max − mean) deviation exceeding that of the Baseline by 6.1%. In contrast, by leveraging UNR-based NIC offloading and computation-communication overlap, POLAR-PIC effectively masks synchronization overheads. This design reduces

Table 2: Summary of $T_{Particle}$, PPS, CPP in Uniform Plasma at representative PPC points ($u_{th} = 0.01$).

PPC	Scheme	$T_{Particle}$ (s)	PPS (Gparticles/s)	CPP (cycles/particle)	Speedup (×)
1	**Baseline (WarpX)**	**1.102**	**0.381**	**3.416**	**1.00**
	Matrix-PIC	1.852	0.226	5.741	0.60
	POLAR-PIC	1.379	0.304	4.275	0.80
8	Baseline (WarpX)	4.278	0.784	1.657	1.00
	Matrix-PIC	2.290	1.465	0.887	1.87
	POLAR-PIC	**1.769**	**1.896**	**0.686**	**2.42**
64	Baseline (WarpX)	30.280	0.887	1.466	1.00
	Matrix-PIC	9.798	2.740	0.475	3.09
	POLAR-PIC	**3.750**	**7.158**	**0.182**	**8.07**
512	Baseline (WarpX)	236.939	0.906	1.434	1.00
	Matrix-PIC	101.673	2.112	0.615	2.33
	POLAR-PIC	**21.651**	**9.919**	**0.131**	**10.94**

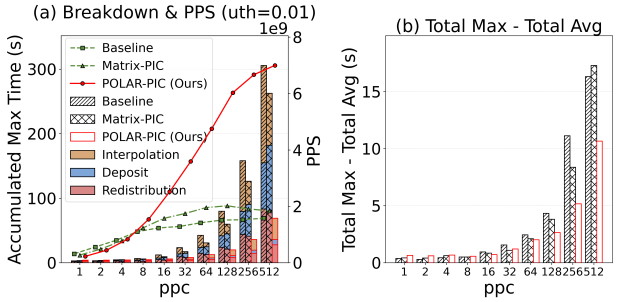


Figure 7: Particle-Phase Performance and Long-tail Effect Analysis of Laser-Ion Acceleration Simulations with Varying PPC.

the cumulative long-tail latency by 34.6% compared to the Baseline and 38.3% compared to Matrix-PIC, confirming that POLAR-PIC delivers not just higher throughput, but predictable stability essential for highly dynamic production simulations.

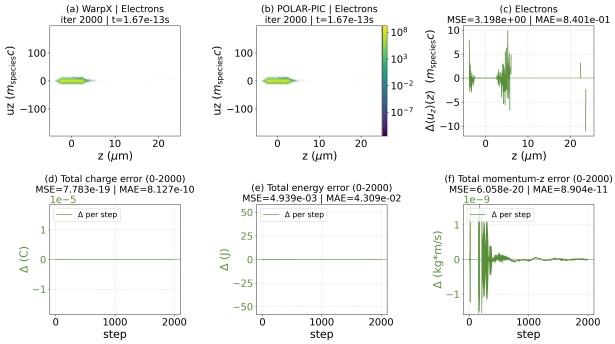


Figure 8: Correctness verification for Laser-Ion Acceleration at step 2000 and over the full 2000-step evolution.

6.1.3 Correctness Verification for Laser-Ion Acceleration. To ensure numerical fidelity, we conducted a controlled comparison against the native WarpX baseline on the LX2 platform using the LIA benchmark. Figure 8 reports a qualitative phase-space comparison at step 2000, together with the per-step errors of total charge, total energy, and longitudinal momentum over 2000 steps. As illustrated in Figure 8, the electron longitudinal phase-space structures and beam morphologies remain highly consistent between native WarpX and POLAR-PIC. Quantitative phase-space assessment yields a Mean Squared Error (MSE) of 3.198 and a Mean Absolute Error (MAE) of 0.8401 (normalized to $m_e c$). Given that the peak momentum exceeds 100 $m_e c$, the MAE corresponds to a relative error below 0.84%. The remaining discrepancies are primarily attributed to the altered floating-point summation order inherent in the matrix outer-product reformulation. Meanwhile, the per-step error curves in Figure 8(d–f) show that the total charge, total energy, and longitudinal momentum errors remain stably bounded throughout the

2000-step evolution. Together, these results indicate that POLAR-PIC preserves numerical consistency without introducing visible artifacts or instability in this benchmark.

6.2 Ablation 1: Interpolation Kernel Evolution

In these experiments, we fix the Deposition strategy to the native WarpX deposition path (D0) and disable communication overlap to isolate the impact of data supply strategies and kernel formulations.

6.2.1 Stability of SoW Data Supply (G0–G4). We first evaluate the stability of data supply strategies under varying migration intensities, finding that the SoW mechanism (G4) sustains high throughput (5.2×10^9 PPS) even at $u_{th} = 0.2$.

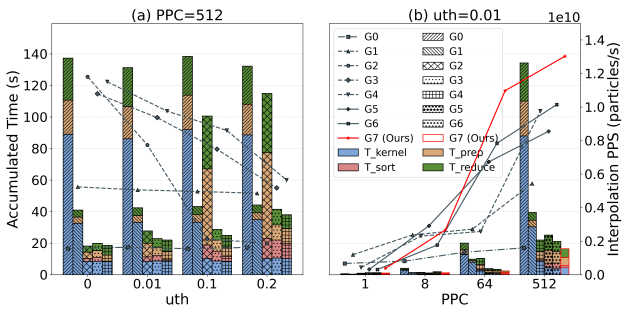


Figure 9: VPU results for G0–G4 at $PPC = 512$, and MPU Interpolation results for G5–G7 across PPC at $u_{th} = 0.01$ (with G0/G1 shown as baselines).

As shown in Figure 9(a), while unsorted baselines (G0/G1) are insensitive to dynamics due to their low-throughput slow sequential access, logical ordering (G2) provides benefits only in low-migration regimes ($u_{th} < 0.1$). As disorder intensifies, the indirect memory accesses in G2 incur escalating address-generation overheads. Specifically,T_{prep} and T_{reduce} increase by 5.6× as u_{th} rises from 0.01 to 0.1, neutralizing the benefits of logical grouping. Conversely, G3 and G4 demonstrate that physical continuity is critical for sustaining bandwidth. Although G3 improves locality via index-guided write-back, its sorting overhead is 1.19× higher than G4's tail-sorting approach at $u_{th} = 0.2$. By implementing the dual-region SoW mechanism, G4 eliminates redundant sorting for resident particles, maintaining optimal data supply bandwidth across all dynamic regimes.

6.2.2 Impact of Data Supply on MPU Interpolation (G0–G1, G5–G7). Fixing migration at $u_{th} = 0.01$, we sweep particle density to demonstrate that physically contiguous data supply is the prerequisite for unlocking the full acceleration potential of the matrix outer-product operator.

Figure 9(b) reveals that in the ultra-sparse regime ($PPC = 1$), MPU variants lag due to unamortized padding and metadata overheads (> 80% of runtime). However, as density increases ($PPC \geq 8$), the outer-product operator delivers substantial gains. G5 improves the kernel-stage throughput by 18.5× over G0 at $PPC = 512$, and introducing physical reordering (G6) further boosts memory bandwidth utilization by 1.2×. The fully realized G7, which combines

Table 3: Performance summary for Interpolation (G0–G7) and Deposition (D0–D3) variants under representative conditions (PPC=512, $u_{th} = 0.01$)

Variant	Time (s)	PPS (Gparticle/s)	CPP (cycles/particle)	Speedup (×)
Interpolation & Push				
G0	131.312	1.635	0.795	1.00
G1	42.476	5.056	0.257	3.09
G2	27.778	7.731	0.168	4.73
G3	22.936	9.363	0.139	5.73
G4	22.013	9.756	0.133	5.97
G5	25.121	8.548	0.152	5.23
G6	21.178	10.140	0.128	6.20
G7	**16.490**	**13.023**	**0.100**	**7.96**
Deposition				
D0	86.221	2.491	0.522	1.00
D1	51.142	4.199	0.310	1.69
D2	6.922	31.023	0.042	12.46
D3	**6.523**	**32.922**	**0.039**	**13.22**

Tail Sorting with the *Stream-Split* mechanism, achieves the optimal synergy between data layout and hardware characteristics. As detailed in Table 3, G7 reduces the computational cost to a minimal 0.100 CPP, representing a 7.96× efficiency gain over the Baseline. These results confirm that while the MPU reformulation represents a significant architectural enhancement, the SoW-based data supply is the necessary enabler for its efficiency.

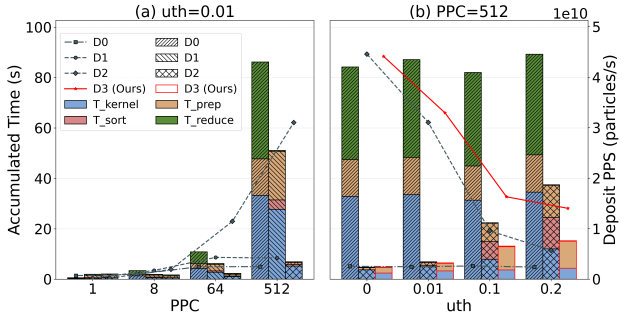


Figure 10: Reuse benefits for D0–D2 under fixed $u_{th} = 0.01$ and Robustness for D0, D2-D3 under PPC=512.

6.3 Ablation 2: Synergistic Gains in Deposition

These experiments fix the Interpolation operator to either G2 (logical) or G4 (physical) and vary Deposition implementations (D0–D3) to evaluate the efficiency of layout reuse.

6.3.1 Comparison of Different Layout Reuse (D0–D2). We evaluate layout reuse efficiency, finding that explicitly reusing the physically contiguous layout (D2) significantly outperforms logical ordering (D1), accelerating preparation and kernel execution by 23× and 5.3×, respectively, at $PPC = 512$.

Figure 10(a) shows that while both D1 and D2 mitigate atomic write conflicts compared to the baseline D0, the advantage of physical contiguity in D2 is decisive. Crucially, since D2 reuses the dual-region SoW structure and requires only lightweight sorting of

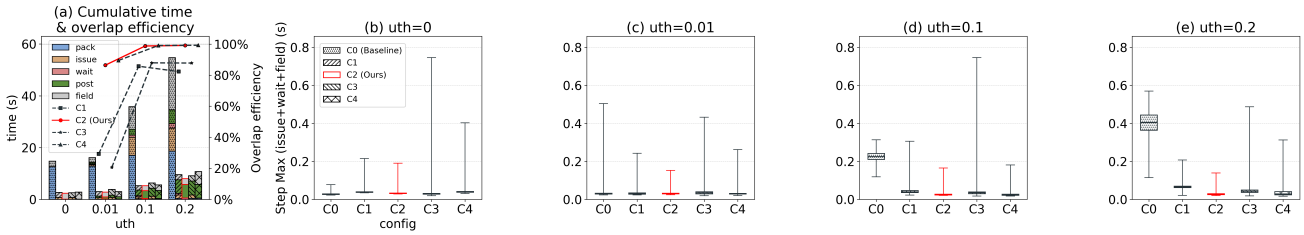


Figure 11: Impact of u_{th} on overlap efficiency and runtime breakdown. Comparisons between average and maximum rank times highlight load imbalance and tail latency effects.

the disordered tail, its sorting cost is reduced by 83.5% relative to D1. This architectural efficiency is quantified in Table 3: while logically ordered variants consume > 0.3 CPP due to atomic contention and index maintenance, the physically contiguous D2 effectively saturates Deposition bandwidth, reducing the cost to ∼ 0.04 CPP.

6.3.2 Trade-off Between Dynamic–Static Splitting and Tail Over-Sorting (D0, D2–D3). Comparing tail handling strategies under varying migration intensities reveals that the dynamic–static splitting policy (D3) yields the optimal trade-off, avoiding the 33% sorting overhead incurred by forced reordering (D2) at high turbulence.

We fix $PPC = 512$ and sweep u_{th} to evaluate robustness. As shown in Figure 10(b), D2 and D3 are comparable in low-migration regimes. However, as migration intensity increases, D2 suffers from escalating sorting costs and degraded MPU data supply efficiency for the resorted tail. At $u_{th} = 0.2$, D2's kernel performance lags behind D3 by 2.7×. By falling back to the VPU atomic path only for the sparse disordered tail, D3 avoids expensive re-sorting while maintaining high-throughput MPU execution for the majority of particles, which reside in the Ordered Region. This confirms that reusing the physically contiguous layout combined with a hybrid splitting policy constitutes the robust Deposition pipeline even under worst-case migration scenarios.

6.4 Ablation 3: Benefits of Overlap

6.4.1 Overlap Efficiency and Hardware Offloading (C0–C2). Evaluating communication efficiency under dynamic loads ($PPC = 512$, varying u_{th}) reveals that the synergistic combination of SoW-based pre-packing and RDMA offloading (C2) eliminates explicit synchronization barriers, achieves a peak overlap ratio of 99.1%, and reduces total communication time by 20× compared to the MPI-based alternative.

As illustrated in Figure 11(a), the foundation of this efficiency is the SoW mechanism, which eliminates redundant traversals and reduces the exposed standalone scan-and-pack stage by ≥ 99% across all migration rates. Importantly, this does not mean that packing work vanishes entirely. Instead, the residual packing cost is absorbed into the write-back path described in Section 4.4.1 and is therefore accounted for within $T_{Interpolation}$ rather than as a separate end-of-step stage. As indicated by the measurements summarized in Figure 9(a), the packing cost T_{Sort} absorbed into the write-back store path contributes only 0.146 s (3.3% of $T_{Interpolation}$) to $T_{Interpolation}$ under the representative case of $u_{th} = 0.01$, whereas the exposed standalone packing stage in the baseline accounts for 12.869 s. Building on this fusion, the computation–communication pipeline further amplifies the gains. While the MPI-based C1 reaches a saturation point of 82.3% overlap due to software overheads, the UNR-based C2 leverages RDMA hardware offloading to fully decouple data transmission from CPU intervention. This near-perfect overlap effectively masks redistribution costs. Specifically, C2 reduces the maximum per-rank wait time to just 0.01 s, representing a 58× reduction relative to C1, which verifies that the asynchronous design successfully hides synchronization latency behind the Deposition kernel.

6.4.2 Stability and Network Contention (C0–C4). Analyzing the stability of the communication-critical path ($T_{issue} + T_{wait} + T_{field}$) via boxplots reveals that the conservative overlap strategy (C2) provides the most robust time-to-solution by avoiding the network contention inherent in aggressive overlap strategies (C3/C4).

As shown in Figure 11(b–e), while the bulk-synchronous baseline (C0) suffers a scalability collapse with a > 14× latency surge under dynamic loads ($u_{th} = 0.2$), asynchronous strategies generally mitigate this impact. However, attempting to maximize overlap by extending particle traffic into the field-solver phase (C3/C4) proves counterproductive. Specifically, although the MPI-based C3 lowers median latency, it induces severe jitter, increasing maximum latency by 2.3× compared to C1. This contention persists even with hardware offloading: the aggressive C4 exhibits a 2.25× longer tail compared to the conservative C2, despite having comparable medians. This degradation confirms that extending particle traffic saturates NIC bandwidth, interfering with latency-sensitive field synchronization. By isolating communication to the Deposition window, our method (C2) eliminates these resource conflicts, maintaining the tightest latency distribution and ensuring predictable time-to-solution.

6.5 Cross-Platform Peak Efficiency

For contextual reference, POLAR-PIC achieves 13.2% of theoretical peak efficiency on the LX2 CPU, while WarpX reaches 9.6% on the NVIDIA A800 GPU. On the evaluated LX2 platform, this corresponds to a 13.2× improvement in peak-efficiency ratio over the native WarpX port.

As detailed in Table 4, the native WarpX reference on LX2 is constrained to ∼ 1% efficiency due to irregular memory access and VPU throughput ceilings, whereas POLAR-PIC substantially improves utilization through outer-product reformulation and sustained data continuity. On the evaluated matrix-centric CPU platform, this

domain-specific co-optimization yields a 2.4× gain over Matrix-PIC and achieves a node-level Figure of Merit (FOM_{node}) of 1.1×10^{10}. We report the cross-platform comparison only as context, since the platforms and software stacks differ.

Table 4: Cross-platform performance comparison (double precision; normalized to platform peak and reported for context).

Platform	Variant	η_{peak} (%)	FOM_{node}
LX2 CPU (This work)*	WarpX (Native)	1.0	8.3e8
	Matrix-PIC	5.5	3.6e9
	POLAR-PIC	**13.2**	**1.1e10**
NVIDIA A800 (Measured)*	WarpX (CUDA)	9.6	3.3e9
Reference Platforms**	WarpX (Perlmutter A100)	12.9	9.2e8
	WarpX (Summit V100)	8.3	8.0e8
	WarpX (Frontier MI250X)	3.3	1.3e9
	WarpX (Fugaku A64FX)	1.1	2.2e7

* η_{peak} is normalized to the theoretical peak performance of the target hardware.
** Reference values from [17] are provided for context.

6.6 Weak Scalability at Scale

In the end-to-end full-timestep weak-scaling study from 1 to 4,096 nodes (over 2 million physical cores) under high-turbulence stress ($u_{th} = 0.2$), POLAR-PIC maintains a robust weak-scaling efficiency of 67.5%, significantly outperforming the native WarpX reference, which drops to 42.5% due to unmasked communication overheads.

Figure 12 reveals the divergence in end-to-end scaling behavior. While both implementations exhibit scalability exceeding 90% up to 16 nodes, the native WarpX reference degrades sharply at scale because blocking communication fails to hide the growing latency of large-scale interconnects and heavy particle redistribution overheads. In contrast, POLAR-PIC's asynchronous pipeline effectively masks these overheads, sustaining near-ideal scaling (~ 100%) for the particle-handling components (Interpolation, Deposition, and Particle Redistribution) even at the 2-million-core scale, whereas the native WarpX particle efficiency drops to 62.8%. These results validate that our co-design substantially alleviates the particle-processing bottleneck. However, the end-to-end breakdown also reveals an Amdahl's Law implication: once particle costs are reduced, the Field Solver becomes the dominant residual cost, identifying the field-update phase as the critical target for future optimization.

7 Conclusion and Future Work

This paper presents **POLAR-PIC**, a holistic co-design framework that systematically resolves the architectural mismatches between traditional PIC algorithms and emerging matrix-centric hardware. By synergizing the reformulation of Field Interpolation into an MPU-friendly outer-product kernel, the enforcement of physical memory contiguity via the Sort-on-Write (SoW) mechanism, and the deployment of an RMA-based asynchronous communication pipeline, POLAR-PIC establishes a highly efficient integrated execution path for particle simulations.

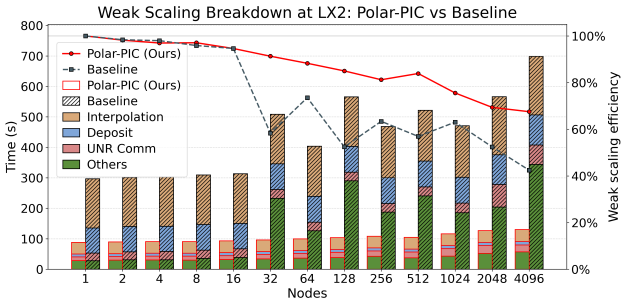


Figure 12: End-to-end full-timestep weak-scaling breakdown from 1 to 4096 nodes (over 2 million cores) on the LS system under high-turbulence stress ($PPC = 512,\ u_{th} = 0.2$).

Experimental evaluations on the next-generation LS pilot system demonstrate that POLAR-PIC significantly improves the particle-processing path over existing solutions. In uniform plasma benchmarks, it achieves speedups of up to 10.9× over the native WarpX reference pipeline on LX2 and outperforms the matrix-based state-of-the-art Matrix-PIC by 4.7×. Extending to real-world laser-ion acceleration scenarios, the framework maintains robust performance, achieving speedups of 4.4× over the native WarpX reference and 3.8× over Matrix-PIC despite intense particle migration. Moreover, the asynchronous communication design effectively masks redistribution overhead, sustaining an overlap ratio of 99.1% even under high-turbulence conditions. For contextual reference, POLAR-PIC attains 13.2% of theoretical peak efficiency on the evaluated CPU-based LS system, while WarpX reaches 9.6% on NVIDIA A800 GPUs. Furthermore, the framework maintains 67.5% end-to-end weak scaling efficiency from 1 to 4,096 nodes (aggregating over 2 million cores).

With the particle-processing bottleneck effectively mitigated, our end-to-end weak-scaling analysis indicates that the field solver emerges as the new dominant cost in the simulation time step. Consequently, future work will focus on optimizing field-solver algorithms and their associated communication paths to better match the throughput of the accelerated particle kernels. Additionally, we plan to extend the POLAR-PIC paradigm to support more complex multi-physics scenarios, such as Quantum Electrodynamics (QED) and dynamic load balancing in non-uniform mesh refinements, to address a broader spectrum of high-energy physics challenges.

Acknowledgments

We sincerely thank all the anonymous reviewers for their valuable feedback. This research was supported by Guangdong S&T Program under Grant No. 2024B0101040005, the National Natural Science Foundation of China (NSFC): No.62461146204 and No.62502552, Guangdong Province Special Support Program for Cultivating High-Level Talents: 2021TQ06X160, National Key Research and Development Program of China under Grant No. YFE030170000 and the Strategic Priority Research Program of Chinese Academy of Sciences under Grant No. XDB0790203.

This research used the open-source particle-in-cell code WarpX. We acknowledge all WarpX contributors.

References

[1] Tony D Arber, Keith Bennett, Christopher S Brady, Alistair Lawrence-Douglas, MG Ramsay, Nathan J Sircombe, Paddy Gillies, Roger G Evans, Holger Schmitz, Anthony R Bell, et al. 2015. Contemporary particle-in-cell approach to laser-plasma modelling. *Plasma Physics and Controlled Fusion* 57, 11 (2015), 113001.

[2] Vignesh Balaji and Brandon Lucia. 2019. Combining Data Duplication and Graph Reordering to Accelerate Parallel Graph Processing. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing* (Phoenix, AZ, USA) *(HPDC '19)*. Association for Computing Machinery, New York, NY, USA, 133–144. doi:10.1145/3307681.3326609

[3] Yann Barsamian, Arthur Charguéraud, Sever A. Hirstoaga, and Michel Mehrenberger. 2018. Efficient Strict-Binning Particle-in-Cell Algorithm for Multi-core SIMD Processors. In *Euro-Par 2018: Parallel Processing*, Marco Aldinucci, Luca Padovani, and Massimo Torquati (Eds.). Springer International Publishing, Cham, 749–763.

[4] Yann Barsamian, Sever A. Hirstoaga, and Éric Violard. 2018. Efficient data layouts for a three-dimensional electrostatic Particle-in-Cell code. *Journal of Computational Science* 27 (2018), 345–356. doi:10.1016/j.jocs.2018.06.004

[5] A. Beck, J. Derouillat, M. Lobet, A. Farjallah, F. Massimo, I. Zemzemi, F. Perez, T. Vinci, and M. Grech. 2019. Adaptive SIMD optimizations in particle-in-cell codes with fine-grain particle sorting. *Computer Physics Communications* 244 (2019), 246–263. doi:10.1016/j.cpc.2019.05.001

[6] Michael A. Bender and Haodong Hu. 2007. An adaptive packed-memory array. *ACM Trans. Database Syst.* 32, 4 (Nov. 2007), 26–es. doi:10.1145/1292609.1292616

[7] Robert Bird, Nigel Tan, Scott V. Luedtke, Stephen Lien Harrell, Michela Taufer, and Brian Albright. 2022. VPIC 2.0: Next Generation Particle-in-Cell Simulations. *IEEE Transactions on Parallel and Distributed Systems* 33, 4 (2022), 952–963. doi:10.1109/TPDS.2021.3084795

[8] Jay P Boris. 1970. Relativistic plasma simulation-optimization of a hybrid code. In *Proc. Fourth Conf. Num. Sim. Plasmas*. 3–67.

[9] Hang Cao, Liang Yuan, He Zhang, Yunquan Zhang, Baodong Wu, Kun Li, Shigang Li, Minghua Zhang, Pengqi Lu, and Junmin Xiao. 2023. AGCM-3DLF: Accelerating Atmospheric General Circulation Model via 3-D Parallelization and Leap-Format. *IEEE Transactions on Parallel and Distributed Systems* 34, 3 (2023), 766–780. doi:10.1109/TPDS.2022.3231013

[10] Yuetao Chen, Kun Li, Yuhao Wang, Donglin Bai, Lei Wang, Lingxiao Ma, Liang Yuan, Yunquan Zhang, Ting Cao, and Mao Yang. 2024. ConvStencil: Transform Stencil Computation to Matrix Multiplication on Tensor Cores. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Edinburgh, United Kingdom) *(PPoPP '24)*. Association for Computing Machinery, New York, NY, USA, 333–347. doi:10.1145/3627535.3638476

[11] Joong-Yeon Cho, Pu-Rum Seo, and Hyun-Wook Jin. 2023. Exploiting copy engines for intra-node MPI collective communication. *The Journal of Supercomputing* 79, 16 (2023), 17962–17982.

[12] S. Cools and W. Vanroose. 2017. The communication-hiding pipelined BiCGstab method for the parallel solution of large unsymmetric linear systems. *Parallel Comput.* 65 (2017), 1–20. doi:10.1016/j.parco.2017.04.005

[13] Viktor K. Decyk and Tajendra V. Singh. 2014. Particle-in-Cell algorithms for emerging computer architectures. *Computer Physics Communications* 185, 3 (2014), 708–719. doi:10.1016/j.cpc.2013.10.013

[14] Alexandre Denis, Julien Jaeger, Emmanuel Jeannot, and Florian Reynier. 2022. A methodology for assessing computation/communication overlap of MPI nonblocking collectives. *Concurrency and Computation: Practice and Experience* 34, 22 (2022), e7168. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.7168 doi:10.1002/cpe.7168

[15] J. Derouillat, A. Beck, F. Pérez, T. Vinci, M. Chiaramello, A. Grassi, M. Flé, G. Bouchard, I. Plotnikov, N. Aunai, J. Dargent, C. Riconda, and M. Grech. 2018. Smilei : A collaborative, open-source, multi-purpose particle-in-cell code for plasma simulation. *Computer Physics Communications* 222 (2018), 351–373. doi:10.1016/j.cpc.2017.09.024

[16] Yusuke Endo, Satoshi Ohshima, and Takeshi Nanri. 2026. Optimization of a GEMM Implementation using Intel AMX. In *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia '26)*. Association for Computing Machinery, New York, NY, USA, 81–90. doi:10.1145/3773656.3773660

[17] Luca Fedeli, Axel Huebl, France Boillod-Cerneux, Thomas Clark, Kevin Gott, Conrad Hillairet, Stephan Jaure, Adrien Leblanc, Rémi Lehe, Andrew Myers, Christelle Piechurski, Mitsuhisa Sato, Neïl Zaim, Weiqun Zhang, Jean-Luc Vay, and Henri Vincenti. 2022. Pushing the Frontier in the Design of Laser-Based Electron Accelerators with Groundbreaking Mesh-Refined Particle-In-Cell Simulations on Exascale-Class Supercomputers. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12. doi:10.1109/SC41404.2022.00008

[18] Guangnan Feng, Jiabin Xie, Dezun Dong, and Yutong Lu. 2024. UNR: Unified Notifiable RMA Library for HPC. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (Atlanta, GA, USA) *(SC '24)*. IEEE Press, Article 105, 15 pages. doi:10.1109/SC41406.2024.00111

[19] Ricardo A Fonseca, Luis O Silva, Frank S Tsung, Viktor K Decyk, Wei Lu, Chuang Ren, Warren B Mori, Shaogui Deng, Shiyoun Lee, T Katsouleas, et al. 2002. OSIRIS: A three-dimensional, fully relativistic particle in cell code for modeling plasma based accelerators. In *International conference on computational science*. Springer, 342–351.

[20] Ping Gao, Xiaohui Duan, Bertil Schmidt, Wusheng Zhang, Lin Gan, Haohuan Fu, Wei Xue, Weiguo Liu, and Guangwen Yang. 2022. Optimization of Reactive Force Field Simulation: Refactor, Parallelization, and Vectorization for Interactions. *IEEE Transactions on Parallel and Distributed Systems* 33, 2 (2022), 359–373. doi:10.1109/TPDS.2021.3091408

[21] Nicolas Guidotti, Pedro Ceyrat, João Barreto, José Monteiro, Rodrigo Rodrigues, Ricardo Fonseca, Xavier Martorell, and Antonio J. Peña. 2021. Particle-In-Cell Simulation Using Asynchronous Tasking. In *Euro-Par 2021: Parallel Processing*, Leonel Sousa, Nuno Roma, and Pedro Tomás (Eds.). Springer International Publishing, Cham, 482–498.

[22] Torsten Hoefler and Andrew Lumsdaine. 2008. Overlapping Communication and Computation with High Level Communication Routines. In *2008 Eighth IEEE International Symposium on Cluster Computing and the Grid (CCGRID)*. 572–577. doi:10.1109/CCGRID.2008.15

[23] Changwan Hong, Aravind Sukumaran-Rajam, Bortik Bandyopadhyay, Jinsung Kim, Süreyya Emre Kurt, Israt Nisa, Shivani Sabhlok, Ümit V. Çatalyürek, Srinivasan Parthasarathy, and P. Sadayappan. 2018. Efficient sparse-matrix multi-vector product on GPUs. In *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing* (Tempe, Arizona) *(HPDC '18)*. Association for Computing Machinery, New York, NY, USA, 66–79. doi:10.1145/3208040.3208062

[24] Han Huang, Jiabin Xie, Guangnan Feng, Xianwei Zhang, Dan Huang, Zhiguang Chen, and Yutong Lu. 2025. HStencil: Matrix-Vector Stencil Computation with Interleaved Outer Product and MLA. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, New York, NY, USA, 1816–1829. doi:10.1145/3712285.3759769

[25] Jiajun Huang, Kaiming Ouyang, Yujia Zhai, Jinyang Liu, Min Si, Ken Raffenetti, Hui Zhou, Atsushi Hori, Zizhong Chen, Yanfei Guo, and Rajeev Thakur. 2023. Accelerating MPI Collectives with Process-in-Process-based Multi-object Techniques. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (Orlando, FL, USA) *(HPDC '23)*. Association for Computing Machinery, New York, NY, USA, 333–334. doi:10.1145/3588195.3595955

[26] Xiaoyan Liu, Yi Liu, Hailong Yang, Jianjin Liao, Mingzhen Li, Zhongzhi Luan, and Depei Qian. 2022. Toward accelerated stencil computation by adapting tensor core unit on GPU. In *Proceedings of the 36th ACM International Conference on Supercomputing* (Virtual Event) *(ICS '22)*. Association for Computing Machinery, New York, NY, USA, Article 28, 12 pages. doi:10.1145/3524059.3532392

[27] A. Myers, A. Almgren, L.D. Amorim, J. Bell, L. Fedeli, L. Ge, K. Gott, D.P. Grote, M. Hogan, A. Huebl, R. Jambunathan, R. Lehe, C. Ng, M. Rowan, O. Shapoval, M. Thévenet, J.-L. Vay, H. Vincenti, E. Yang, N. Zaïm, W. Zhang, Y. Zhao, and E. Zoni. 2021. Porting WarpX to GPU-accelerated platforms. *Parallel Comput.* 108 (2021), 102833. doi:10.1016/j.parco.2021.102833

[28] Yizhuo Rao, Xingjian Cui, Jiabin Xie, Shangzhi Pang, Guangnan Feng, Jinhui Wei, Zhiguang Chen, and Yutong Lu. 2026. Matrix-PIC: Harnessing Matrix Outer-product for High-Performance Particle-in-Cell Simulations. In *In 21st European Conference on Computer Systems (EUROSYS '26),*. Association for Computing Machinery. https://arxiv.org/abs/2601.08277 https://arxiv.org/abs/2601.08277.

[29] Stefan Remke and Alexander Breuer. 2025. Hello SME! Generating Fast Matrix Multiplication Kernels Using the Scalable Matrix Extension. In *Proceedings of the SC '24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis* (Atlanta, GA, USA) *(SC-W '24)*. IEEE Press, 1443–1454. doi:10.1109/SCW63240.2024.00185

[30] Matthieu Schaller, Pedro Gonnet, Aidan B. G. Chalk, and Peter W. Draper. 2016. SWIFT: Using Task-Based Parallelism, Fully Asynchronous Communication, and Graph Partition-Based Domain Decomposition for Strong Scaling on more than 100,000 Cores. In *Proceedings of the Platform for Advanced Scientific Computing Conference* (Lausanne, Switzerland) *(PASC '16)*. Association for Computing Machinery, New York, NY, USA, Article 2, 10 pages. doi:10.1145/2929908.2929916

[31] Patrick Schmid, Maciej Besta, and Torsten Hoefler. 2016. High-Performance Distributed RMA Locks. In *Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing* (Kyoto, Japan) *(HPDC '16)*. Association for Computing Machinery, New York, NY, USA, 19–30. doi:10.1145/2907294.2907323

[32] Shihui Song, Yafan Huang, Peng Jiang, Xiaodong Yu, Weijian Zheng, Sheng Di, Qinglei Cao, Yunhe Feng, Zhen Xie, and Franck Cappello. 2024. CereSZ: Enabling and Scaling Error-bounded Lossy Compression on Cerebras CS-2. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (Pisa, Italy) *(HPDC '24)*. Association for Computing Machinery, New York, NY, USA, 309–321. doi:10.1145/3625549.3658691

[33] Chen Tang, Zhaole Chu, Peiquan Jin, Yongping Luo, and Kuankuan Guo. 2023. HM2: Efficient Host Memory Management for RDMA-Enabled Distributed Systems. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (Orlando, FL, USA) *(HPDC '23)*. Association for Computing Machinery, New York, NY, USA, 335–336. doi:10.1145/3588195.3595952

[34] Didem Unat, Anshu Dubey, Torsten Hoefler, John Shalf, Mark Abraham, Mauro Bianco, Bradford L. Chamberlain, Romain Cledat, H. Carter Edwards, Hal Finkel, Karl Fuerlinger, Frank Hannig, Emmanuel Jeannot, Amir Kamil, Jeff Keasler, Paul H J Kelly, Vitus Leung, Hatem Ltaief, Naoya Maruyama, Chris J. Newburn, and Miquel Pericás. 2017. Trends in Data Locality Abstractions for HPC Systems. *IEEE Transactions on Parallel and Distributed Systems* 28, 10 (2017), 3007–3020. doi:10.1109/TPDS.2017.2703149

[35] J-L Vay. 2008. Simulation of beams or plasmas crossing at relativistic velocity. *Physics of Plasmas* 15, 5 (2008).

[36] Brian Wheatman and Helen Xu. 2021. *A Parallel Packed Memory Array to Store Dynamic Graphs.* SIAM, 31–45. arXiv:https://epubs.siam.org/doi/pdf/10.1137/1.9781611976472.3 doi:10.1137/1.9781611976472.3

[37] Kohji Yoshikawa, Satoshi Tanaka, and Naoki Yoshida. 2021. A 400 trillion-grid Vlasov simulation on Fugaku supercomputer: large-scale distribution of cosmic relic neutrinos in a six-dimensional phase space. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) *(SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 5, 11 pages. doi:10.1145/3458817.3487401

[38] Weiqun Zhang, Ann Almgren, Vince Beckner, John Bell, Johannes Blaschke, Cy Chan, Marcus Day, Brian Friesen, Kevin Gott, Daniel Graves, Max P. Katz, Andrew Myers, Tan Nguyen, Andrew Nonaka, Michele Rosso, Samuel Williams, and Michael Zingale. 2019. AMReX: a framework for block-structured adaptive mesh refinement. *Journal of Open Source Software* 4, 37 (2019), 1370. doi:10.21105/joss.01370

[39] Yiwei Zhang, Kun Li, Liang Yuan, Jiawen Cheng, Yunquan Zhang, Ting Cao, and Mao Yang. 2024. LoRAStencil: Low-Rank Adaptation of Stencil Computation on Tensor Cores. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (Atlanta, GA, USA) *(SC '24)*. IEEE Press, Article 53, 17 pages. doi:10.1109/SC41406.2024.00059

[40] Hui Zhou, Robert Latham, Ken Raffenetti, Yanfei Guo, and Rajeev Thakur. 2024. MPI Progress For All. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 425–435. doi:10.1109/SCW63240.2024.00063

A Implementation Details

Table 5 details the specific implementation strategies for each experimental variant (G0–G7, D0–D3, and C0–C4). It highlights the architectural distinctions between baseline VPU approaches and the proposed MPU-accelerated designs, including the specific sorting mechanisms and overlap protocols employed to isolate performance contributions.

B Simulation Parameters

Table 6 lists the comprehensive physical and numerical parameters used for the Uniform Plasma and Laser-Ion Acceleration benchmarks. These parameters are chosen to ensure the reproducibility of the performance results and to reflect representative workloads in high-energy density physics simulations.

C Artifact Appendix

C.1 Abstract

This artifact provides the open-source implementation of **POLAR-PIC** The released repository is built on top of WarpX and contains the implementation of the main ideas presented in this paper: an MPU-oriented particle-processing path, physically ordered particle layout management, and overlapped particle redistribution.

C.2 Description & Requirements

C.2.1 How to access. The artifact is publicly available at https://github.com/sherry-roar/polarpic-ad.

C.2.2 What is released. The repository releases the implementation basis of POLAR-PIC on top of WarpX. In particular, it includes the code paths corresponding to:

- matrixized / physically ordered particle push,
- custom order-3 deposition,
- communication overlap and one-sided communication integration,
- AMReX-side metadata changes required by physical sorting and incremental sorting.

At the algorithm level, the released implementation reflects the three key ideas of the paper: reformulating Field Interpolation into an MPU-friendly outer-product form, maintaining physical locality through particle layout management, and overlapping particle communication with computation.

C.2.3 Dependencies. The artifact is based on WarpX and retains the corresponding WarpX/AMReX 24.07 implementation context. It also includes a vendored one-sided communication backend used by the released implementation. The best performance reported in the paper is obtained on the LS pilot system with LX2 CPUs and the associated software stack described in the main paper.

C.2.4 Portability. The repository preserves the source-level implementation of the POLAR-PIC optimization path, while some low-level mappings remain platform dependent. In the released code, the `vpu`/`mpu` paths should be understood as placeholders for the target machine's vector/matrix execution path. When porting to another architecture, these parts can be adapted by replacing the corresponding `vpu`/`mpu` instruction functions and related backend-specific support. For example, when targeting Apple M-series processors, the same optimization path can in principle be retargeted by replacing the current `vpu`/`mpu` implementations with the corresponding instruction-level functions on that platform. Likewise, if the original communication backend is unavailable, the communication path can be reimplemented with an equivalent one-sided interface, for example using MPI one-sided communication semantics, provided that the required synchronization behavior is preserved.

C.2.5 Additional note. The released artifact is intended to expose the implementation of POLAR-PIC in an inspectable open-source form. The strongest performance results in this paper correspond to the LS/LX2 platform. If evaluators or readers have questions about the code or platform adaptation, they may contact the authors.

Table 5: Detailed implementations for experimental configurations and ablation variants.

Config	Implementation Specification	Evaluation Purpose & Significance
Exp 1: Interpolation (Gather & Push) Variants		
G0	Native WarpX Gather; relies on compiler auto-vectorization for VPU; no particle ordering.	Establishes the performance baseline under irregular memory access patterns.
G1	Hand-tuned VPU Gather using explicit SIMD intrinsics; unsorted.	Isolates the benefits of manual VPU instruction optimization from data locality factors.
G2	G1 with incremental logical index sorting.	Reproduces the logical-order supply strategy of Matrix-PIC [28]; quantifies locality benefits without physical data movement.
G3	G2 enhanced with physical reordering based on the sorted indices (explicit memory compaction).	Decouples the impact of physical compaction from logical indexing under the VPU kernel.
G4	Replaces G3's reordering with Sort-on-Write (SoW) mechanisms during the write-back path.	Evaluates the efficiency of SoW compared to explicit memory compaction.
G5	MPU Gather kernel enabled; retains G2-style logical index sorting.	Assesses MPU sensitivity to logical-order supply and memory fragmentation (MPU counterpart to G2).
G6	MPU Gather kernel with G3-style physical reordering driven by indices.	Measures the synergistic effect of MPU execution and explicit physical compaction (MPU counterpart to G3).
G7	MPU Gather kernel with SoW physical reordering (Final POLAR-PIC Gather).	Validates that SoW-sustained contiguity enables stable, sustained MPU pipeline saturation.
Exp 2: Deposition (Scatter) Variants		
D0	Native WarpX Deposition; uses default atomic conflict handling strategies.	Baseline for Deposition computational cost and conflict resolution without MPU reformulation.
D1	MPU Deposition reusing G2's incremental logical indices.	Reproduces the Matrix-PIC Deposition strategy [28]; isolates MPU gains under logical ordering.
D2	MPU Deposition reusing G7's physical layout; adds extra binning for the disordered tail stream.	Evaluates whether explicit tail binning improves robustness when perfect global contiguity is absent.
D3	MPU Deposition reusing G7's physical layout; falls back to VPU for the tail stream.	Final design of POLAR-PIC. Maximizes reuse of SoW layout while maintaining efficient, low-overhead tail handling.
Exp 3: Overlap Strategy Variants		
C0	Bulk-synchronous redistribution at end of the step (Scan → Pack → Send → Wait → Unpack).	Baseline BSP behavior; exposes full packing and synchronization latency on the critical path.
C1	MPI-based nonblocking overlap; initiates transfers and converges immediately after Deposit.	Evaluates conservative overlap capabilities using standard MPI progress semantics.
C2	UNR-based notifiable overlap; offloads pre-packed buffers and converges after Deposit.	Default POLAR-PIC strategy. Validates NIC offloading and notifiable completion for stable overlap.
C3	MPI-based overlap extended into the Field Solve phase.	Assesses aggressive overlap limits and potential bandwidth interference with field communication.
C4	UNR-based overlap extended into the Field Solve phase.	Evaluates whether UNR enables deeper overlap windows without destabilizing overall performance.
Integrated Systems		
Baseline	Reference pipeline: (G0 + D0 + C0).	Represents the production-grade WarpX configuration tuned with official optimizations for best-effort performance.
Matrix-PIC	Open-source-based Matrix-PIC architecture (G2 + D1 + C0) reconstructed by reusing incremental index sorting logic.	Extends the matrixized SOTA by applying logical index-sorting to both Interpolation and Deposition without SoW physical reordering. Represents the manually implemented best-case baseline SOTA without SoW.
POLAR-PIC	Proposed pipeline: (G7 + D3 + C2).	Integrates full MPU execution, SoW-sustained contiguity, and UNR-based overlap.

Table 6: Key parameters for Uniform Plasma and Laser-Ion Acceleration workloads.

Input Parameter	Uniform Plasma	Laser-Ion Acceleration
Objective	Assess SIMD/MPU kernel efficiency under controlled conditions	Evaluate system-level behavior under strongly non-uniform, migration-heavy dynamics
Physics Context	Homogeneous periodic plasma for isolating per-step kernel behavior	Laser–solid interaction capturing transient acceleration dynamics
Simulation Configuration		
`max_step`	100	100
`geometry.dims`	3	3
`warpx.grid_type`	collocated	collocated
`amr.n_cell`	$256 \times 128 \times 128$	$192 \times 192 \times 256$
`geometry.prob_lo`	-2.0×10^{-5} (all dims)	$-3.75 \times 10^{-6}, -3.75 \times 10^{-6}, -2.5 \times 10^{-6}$
`geometry.prob_hi`	$+2.0 \times 10^{-5}$ (all dims)	$+3.75 \times 10^{-6}, +3.75 \times 10^{-6}, +1.25 \times 10^{-5}$
Boundary Conditions & Solvers		
`boundary.field`	`periodic` (all faces)	`pml` (all faces)
`warpx.cfl`	1.0	0.999
`algo.particle_shape`	3 (Cubic)	3 (Cubic)
`algo.maxwell_solver`	Yee	Yee
`algo.particle_pusher`	Boris	Boris
Particle Configuration		
`particles.tile_size`	$8 \times 8 \times 8$	$8 \times 8 \times 8$
`electrons.profile`	Constant	`parse_density_function`
`electrons.density`	1.0×10^{25} m^{-3}	–
`electrons.ppc`	{1, 2, 4, …, 512}	{1, 2, 4, …, 512}
`electrons.u_th` (x,y,z)	{0, 0.01, 0.05, 0.1, 0.2}	0.01 (fixed)
`target parameters`	–	$L = 50$ nm, $n_e = 30 n_c$
		$n_c = 1.74 \times 10^{27}$ m^{-3}
Laser Parameters (LIA only)		
`laser.profile`	–	Gaussian
`laser.a0`	–	16.0
`laser.wavelength`	–	0.8 μm
`laser.duration`	–	30 fs
`laser.focal_dist`	–	4.0 μm
`laser.waist`	–	4.0 μm